

Window Functions in SQL

Window functions in SQL are a powerful feature that allows you to perform calculations across a set of table rows that are related to the current row. Unlike aggregate functions (like `SUM()`, `AVG()`, `COUNT()`) which return a single value for a group of rows, window functions return a value for each row in the result set, but the calculation is based on a "window" of rows. This window is defined by the `OVER()` clause.

The `OVER()` Clause

The `OVER()` clause is fundamental to all window functions. It defines the window of rows on which the function operates. It can include three main components:

- **PARTITION BY:** Divides the rows into groups or partitions. The window function is applied independently to each partition. If omitted, the entire result set is treated as a single partition.
- **ORDER BY:** Sorts the rows within each partition. This is crucial for functions that depend on the order of rows, such as `ROW_NUMBER()`, `RANK()`, `LEAD()`, and `LAG()`.
- **ROWS or RANGE:** Defines the frame within each partition. This specifies which rows relative to the current row are included in the window. Common frame specifications include `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (default for many functions), `ROWS BETWEEN N PRECEDING AND N FOLLOWING`, etc.

Types of Window Functions

Window functions can be broadly categorized into several types:

1. Ranking Functions

Ranking functions assign a rank to each row within its partition based on the specified order.

`ROW_NUMBER()`

Assigns a unique, sequential integer to each row within its partition, starting from 1.

Example:

Consider a table `Sales` with `Region` and `SalesAmount`. We want to rank sales within each region.

```
SELECT
Region,
SalesAmount,
ROW_NUMBER() OVER (PARTITION BY Region ORDER BY SalesAmount DESC) AS
SalesRank
FROM
Sales;
```

`RANK()`

Assigns a rank to each row within its partition. If two or more rows have the same value, they receive the same rank, and the next rank is skipped.

Example:

Using the `Sales` table, if we have identical `SalesAmount` values within a `Region`:

```
SELECT

    Region,

    SalesAmount,

    RANK() OVER (PARTITION BY Region ORDER BY SalesAmount DESC) AS SalesRank

FROM

    Sales;
```

`DENSE_RANK()`

Similar to `RANK()`, but it assigns consecutive ranks. If two or more rows have the same value, they receive the same rank, but no ranks are skipped.

Example:

Using the `Sales` table with identical `SalesAmount` values:SELECT

Region,

SalesAmount,

DENSE_RANK() OVER (PARTITION BY Region ORDER BY SalesAmount DESC) AS
SalesRank

FROM

Sales;

NTILE(N)

Divides the rows in each partition into **N** groups (or tiles) and assigns an integer from 1 to **N** indicating the group to which the row belongs.

Example:

Divide employees into 4 quartiles based on their **Salary** within each
Department.SELECT

Department,

EmployeeName,

Salary,

NTILE(4) OVER (PARTITION BY Department ORDER BY Salary DESC) AS SalaryQuartile

FROM

Employees;

2. Analytic Functions (Value Functions)

These functions allow you to access data from a different row within the same result set without self-joining.

LEAD(expression, offset, default)

Returns the value of **expression** from a row that follows the current row by **offset** rows within the partition. If no such row exists, **default** is returned.

Example:

Calculate the sales increase from the previous month. SELECT

SaleDate,

SalesAmount,

LEAD(SalesAmount, 1, 0) OVER (ORDER BY SaleDate) AS NextMonthSales,

(LEAD(SalesAmount, 1, 0) OVER (ORDER BY SaleDate) - SalesAmount) AS
SalesIncrease

FROM

MonthlySales;

LAG(expression, offset, default)

Returns the value of **expression** from a row that precedes the current row by **offset** rows within the partition. If no such row exists, **default** is returned.

Example:

Calculate the sales difference from the previous month. SELECT

SaleDate,

SalesAmount,

LAG(SalesAmount, 1, 0) OVER (ORDER BY SaleDate) AS PreviousMonthSales,

(SalesAmount - LAG(SalesAmount, 1, 0) OVER (ORDER BY SaleDate)) AS
SalesDifference

FROM

MonthlySales;

FIRST_VALUE(expression)

Returns the value of **expression** from the first row in the window frame.

Example:

Get the earliest order date for each customer.SELECT

CustomerID,

OrderDate,

FIRST_VALUE(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS
FirstOrderDate

FROM

Orders;

LAST_VALUE(expression)

Returns the value of **expression** from the last row in the window frame. This often requires explicitly defining the window frame to include subsequent rows.

Example:

Get the latest order date for each customer.SELECT

CustomerID,

OrderDate,

LAST_VALUE(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
LastOrderDate

FROM

Orders;

NTH_VALUE(expression, N)

Returns the value of **expression** from the Nth row in the window frame.

Example:

Find the second highest salary in each department.SELECT

Department,

EmployeeName,

Salary,

NTH_VALUE(Salary, 2) OVER (PARTITION BY Department ORDER BY Salary DESC
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
SecondHighestSalary

FROM

Employees;

3. Aggregate Window Functions

These are standard aggregate functions (`SUM()`, `AVG()`, `COUNT()`, `MIN()`, `MAX()`) used as window functions by adding an `OVER()` clause. This allows them to perform aggregation over a window of rows while still returning a value for each individual row.

`SUM(expression) OVER (...)`

Calculates the sum of `expression` over the specified window.

Example:

Calculate a running total of sales. SELECT

SaleDate,

SalesAmount,

SUM(SalesAmount) OVER (ORDER BY SaleDate ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS RunningTotalSales

FROM

DailySales;

`AVG(expression) OVER (...)`

Calculates the average of `expression` over the specified window.

Example:

Calculate a 7-day moving average of product ratings.SELECT

RatingDate,

Rating,

AVG(Rating) OVER (ORDER BY RatingDate ROWS BETWEEN 6 PRECEDING AND
CURRENT ROW) AS SevenDayMovingAverage

FROM

ProductRatings;

COUNT(expression) OVER (...)

Counts the number of rows or non-NULL values of **expression** over the specified window.

Example:

Count the number of orders each customer has made up to a certain date.SELECT

CustomerID,

OrderDate,

COUNT(OrderID) OVER (PARTITION BY CustomerID ORDER BY OrderDate ROWS
BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS OrdersToDate

FROM

Orders;

4. Distribution Functions

These functions describe the distribution of rows within a window.

CUME_DIST()

Calculates the cumulative distribution of a value within a partition. It represents the proportion of rows with a value less than or equal to the current row's value. The result is between 0 and 1.

Example:

Find the cumulative distribution of employee salaries within each department. SELECT

Department,

EmployeeName,

Salary,

CUME_DIST() OVER (PARTITION BY Department ORDER BY Salary) AS SalaryCumeDist

FROM

Employees;

PERCENT_RANK()

Calculates the percentage rank of a row within its partition. The result is between 0 and 1. The formula is $(\text{RANK}() - 1) / (\text{number of rows in partition} - 1)$.

Example:

Calculate the percentage rank of sales within each product category. SELECT

ProductCategory,

ProductName,

SalesAmount,

PERCENT_RANK() OVER (PARTITION BY ProductCategory ORDER BY SalesAmount
DESC) AS SalesPercentRank

FROM

ProductSales;

Window functions are incredibly versatile and allow for complex analytical queries that would otherwise be difficult or inefficient to achieve using traditional joins and subqueries. Mastering them is key to advanced SQL data analysis.